

# SAYI AVCISI (NUMBER CATCHER)

## 8086 Assembly Dili ile Yazılmış DOS Tabanlı Oyunun Derinlemesine Analizi

### 1. Oyun Tanıtımı

Sayı Avcısı, 16-bit x86 mimarisi üzerinde, doğrudan işlemci kayıtçıları ve BIOS kesmeleri kullanılarak geliştirilmiş retro tarzı bir DOS oyunudur. Bu proje, donanım seviyesinde programlama, bellek yönetimi ve gerçek zamanlı oyun döngüsü mantığını bir araya getirir. Oyunun temel amacı, sınırlı bir süre içerisinde ekrandaki hareketli hedefleri yakalayarak en yüksek skora ulaşmaktır.

**Platform:** DOS / DOSBox (8086 Mimarisi)

**Dil:** x86 Assembly (Intel Syntax)

**Geliştirici:** Emre Turan

Bu döküman, oyunun hem kullanıcı tarafındaki işleyişini hem de arka plandaki teknik kod yapısını, değişken yönetimini ve kullanılan kritik algoritmaları detaylı bir şekilde incelemektedir [cite: 4].

## 2. Oynanış ve Kontroller

Sayı Avcısı, basit ama etkileşimli bir mekaniğe sahiptir. Oyuncu, ekranda beliren kendi sayısını (n2) klavye yardımıyla hareket ettirerek, bilgisayar tarafından otomatik olarak hareket ettirilen hedef sayıyı (n1) yakalamaya çalışır.

### Kontrol Mekanizması

Oyun, standart klavye tuşları ile gerçek zamanlı olarak kontrol edilir:

| Tuş      | Hareket | Açıklama                    |
|----------|---------|-----------------------------|
| <b>W</b> | Yukarı  | Karakteri üst satıra taşır. |
| <b>S</b> | Aşağı   | Karakteri alt satıra taşır. |
| <b>A</b> | Sola    | Karakteri sol sütuna taşır. |
| <b>D</b> | Sağa    | Karakteri sağ sütuna taşır. |

Kontroller, "non-blocking" (beklemesiz) bir yapıda tasarlanmıştır; yani oyuncu bir tuşa basmasa bile oyun akmaya ve süre azalmaya devam eder.

### 3. Oyun Kuralları ve Mekanikler

Oyunun sürdürülebilirliği ve zorluk dengesi aşağıdaki temel kurallar üzerine inşa edilmiştir:

#### Temel Kurallar

- Zaman Sınırı:** Oyuncunun her turda 30 saniyelik bir süresi vardır. Süre sıfırlandığında "OYUN BITTİ!" mesajı görüntülenir.
- Skor Sistemi:** İki sayı çarpıştığında, sayılardaki değerler toplanarak mevcut skora eklenir.
- Dinamik Hedef:** Belirlenen baraj skora ulaşıldığında, hedef puan otomatik olarak 5 puan artırılarak oyunun devamlılığı sağlanır.
- Sayı Üretimi:** Çarpışma sonrası sayılar -9 ile +9 arasında rastgele değerler olarak yer değiştirir.

#### Oyun Alanı Sınırları

Hedef sayı (n1), ekranın belirli bir bölgesinde (X: 32-45, Y: 8-14) sürekli hareket eder ve duvarlara çarptığında yön değiştirir (Bounce mekaniği). Oyuncu ise tüm ekranda serbestçe hareket edebilir.

## 4. Teknik Analiz: Bellek ve Veri Yönetimi

### Bellek Modeli ve Direktifler

Kodun başında yer alan **org 100h** direktifi, programın bir COM dosyası olarak derleneceğini belirtir [cite: 6]. Bu modelde kod, veri ve yığın segmentleri aynı 64 KB'lık alanda bulunur [cite: 7]. Başlangıç ofseti, DOS'un PSP alanı için rezerve ettiği 256 byte'tan sonra (100h) başlar [cite: 8]. **jmp start** komutu ile veri tanımlamaları geçilerek doğrudan koda dallanılır [cite: 9].

### Veri Segmenti

| Değişken [cite: 12] | Boyut [cite: 12] | Başlangıç [cite: 12] | Fonksiyon [cite: 12]          |
|---------------------|------------------|----------------------|-------------------------------|
| x1, y1              | DB               | 35, 10               | Hedefin (n1) koordinatları.   |
| yonX, yonY          | DB               | 1, 1                 | Hareket yön vektörleri.       |
| x2, y2              | DB               | 38, 11               | Oyuncunun (n2) koordinatları. |
| n1, n2              | DB               | 3, 2                 | Render edilen sayı değerleri. |
| skor, hedef         | DB               | 0, 10                | Puan ve baraj skor.           |
| sure                | DB               | 30                   | Geri sayım saniyesi.          |
| lastTick            | DW               | 0                    | Zamanlama için Word tipi.     |

### Donanım Kesmeleri

- **INT 10h:** Ekran temizleme (AH=00h), imleç konumu (AH=02h) ve karakter basımı (AH=0Eh) [cite: 15, 16, 17, 18].

- **INT 1Ah:** Sistem saat vuruşlarını okuyarak süre ve gecikme yönetimi sağlar (Saniyede 18.2 vuruş) [cite: 20, 21, 22].
- **INT 16h:** Klavye tamponunu kontrol eder (AH=01h) ve tuş vuruşlarını okur (AH=00h) [cite: 23, 24, 25, 26].

## 5. Kritik Algoritmalar ve Sonuç

### Çarpışma Algoritması

Maliyetli Öklid mesafesi yerine, her iki eksenindeki mutlak fark kontrol edilir [cite: 29, 30]. Fark 1 veya 0 ise çarpışma kabul edilir [cite: 43].

```
mov al, x1      ; [cite: 31,
37] sub al, x2  ; [cite: 32,
38]
jns xp          ; Pozitifse atla [cite: 33, 39]
neg al         ; Negatifse pozitive çevir [cite: 34, 40] xp:
cmp al, 1      ; Fark <= 1 mi? [cite: 35, 41]
ja p_ui       ; Değilse çarpışma yok [cite: 36, 42]
```

### Rastgele Sayı ve Yazdırma

Rastgele sayı üretimi için sistem saatinin milisaniyesi (DL) kullanılır [cite: 45, 46]. Bu değer 19'a bölünür ve kalan (AH) üzerinden [-9, +9] aralığı oluşturulur [cite: 47, 48, 49, 50]. Sayı yazdırma işlemi (**print\_number**), sayıyı 10'a bölerek basamaklarına ayırır ve her birine 30h ekleyerek ASCII karşılığını ekrana basar [cite: 53, 54, 55, 56].

### Sonuç ve Gelecek Geliştirmeler

Bu çalışma, 16-bit kısıtları altında verimli bir oyun döngüsü ve durum makinesi (State Machine) sunmaktadır [cite: 59]. Gelecekte, ekran kırışmalarını önlemek için **Double Buffering** [cite: 62] ve ekran sınırlarını yönetmek için **Border Collision** [cite: 61] mekanizmaları eklenebilir.